

Modern AI systems: agents and system optimizations

Agent design III: memory, and reliability

Juncheng Yang



Harvard John A. Paulson
School of Engineering
and Applied Sciences



HARVARD UNIVERSITY

MaDSys

Measurement and Design of Systems Lab

Logistics

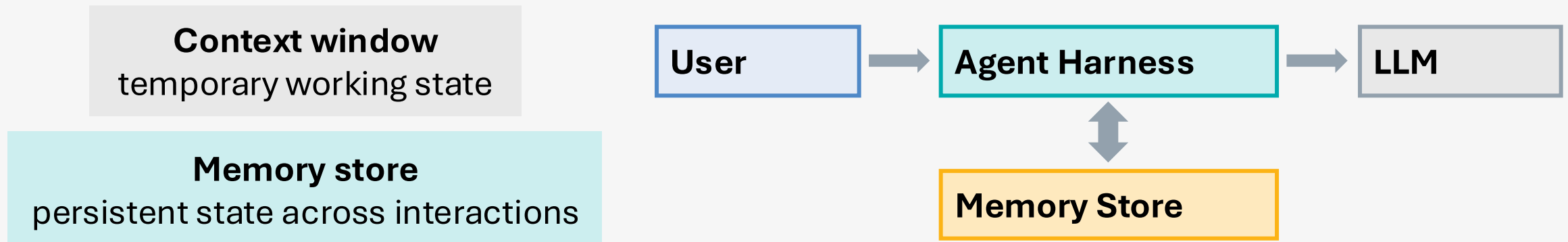
- Office hour this Thursday 3:30pm SEC 4.308
- Final project poster presentation: 9am-1pm Dec 7
 - please reach out this week if the time does not work for you
- AI-generated reference materials
 - <https://cs2680.com/notes/agent-evaluation>
 - notes in the reading (the list is curated by me)
 - reviewed by me and TFs

Plan for today

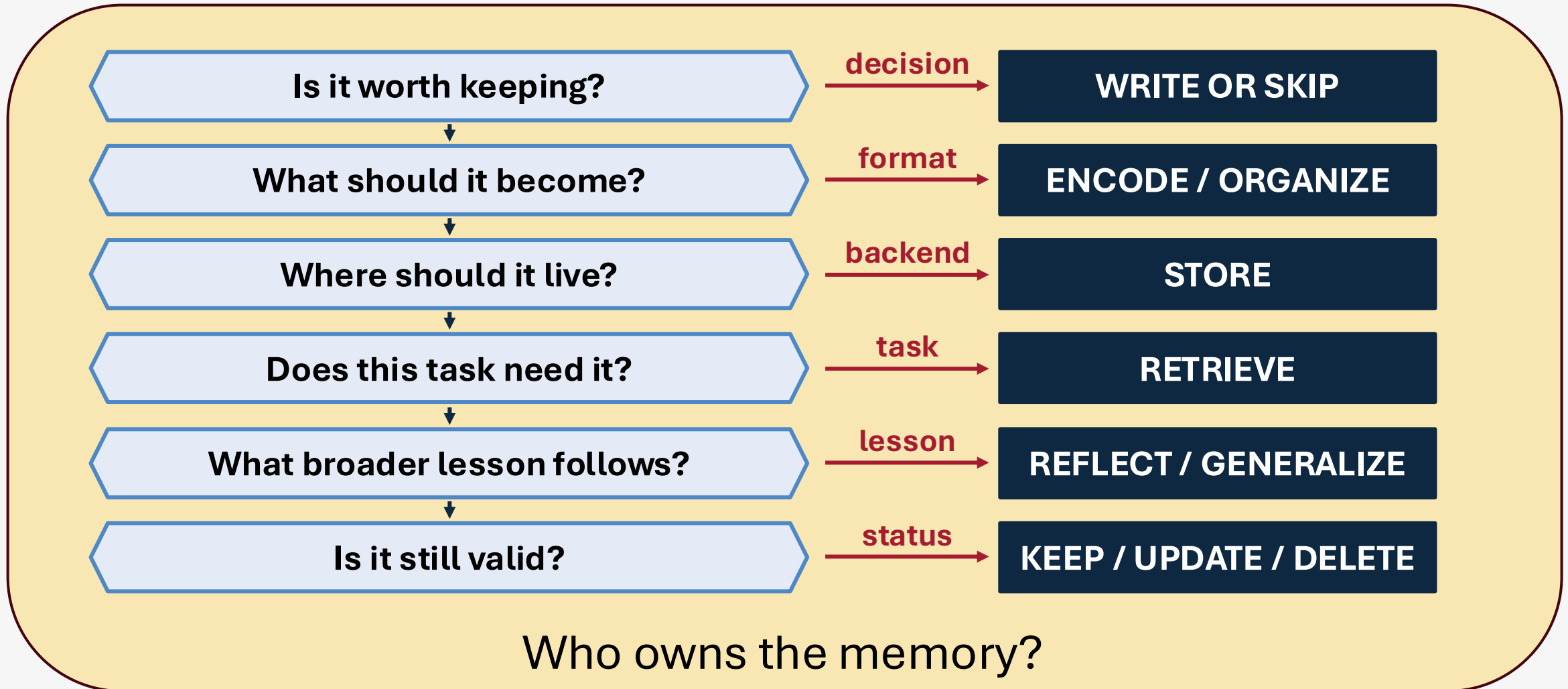
- Multi-agent design (cont from last lecture)
- Agent memory
- Agent reliability

Agent memory

Persistent state beyond the context window



Agent memory design questions



Decision: when should something become memory?

Writing every interaction raises recall and noise

write every interaction

high recall



high precision

selective writing



Explicit user request



Deterministic rules (heuristics)



Small classifier



Periodic LLM review



How to organize the memory?

Raw events / episodes

“We moved serving from vLLM to SGLang.”

High fidelity; large and redundant

Unstructured summaries

“Uses SGLang for model serving.”

Compact, but may lose context

Structured state

`{"serving_engine": "SGLang"...}`

Queryable; schema constrains

Knowledge graph

Project → USES → SGLang

Linked entities and change over time



How to store the memory?

Plain files	CLAUDE.md, scratchpad	Simple to inspect, diff, and search
Key-value or relational database	Redis, SQLite, Postgres JSONB	Exact lookup, filtering, and transactions
Vector store	pgvector, Chroma, Pinecone	Exact or approximate similarity search
Graph database	Neo4j, FalkorDB (Graphiti)	Relationship queries and multi-hop traversal

When memory should be used?

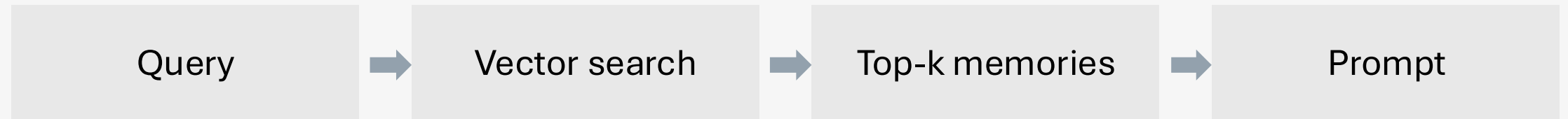
Automatic	Runtime-controlled	Model-controlled
Every turn: search → LLM → add	<pre>if memory_needed: search() if memorable: add()</pre>	<pre>search_memory() add_memory() update_memory() forget_memory()</pre>
+ Predictable and simple	+ Selective and efficient	+ Demand-driven; the LLM decides when to invoke
– Adds latency and irrelevant context	– Need rules or a classifier	– May miss useful reads or writes

Many systems use a hybrid approach; read and write policy may differ

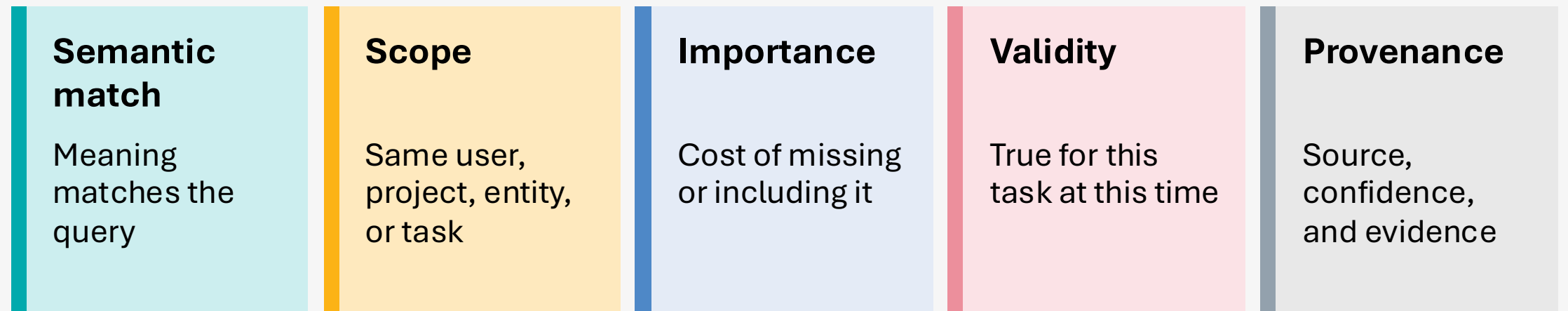


How to retrieve the memory?

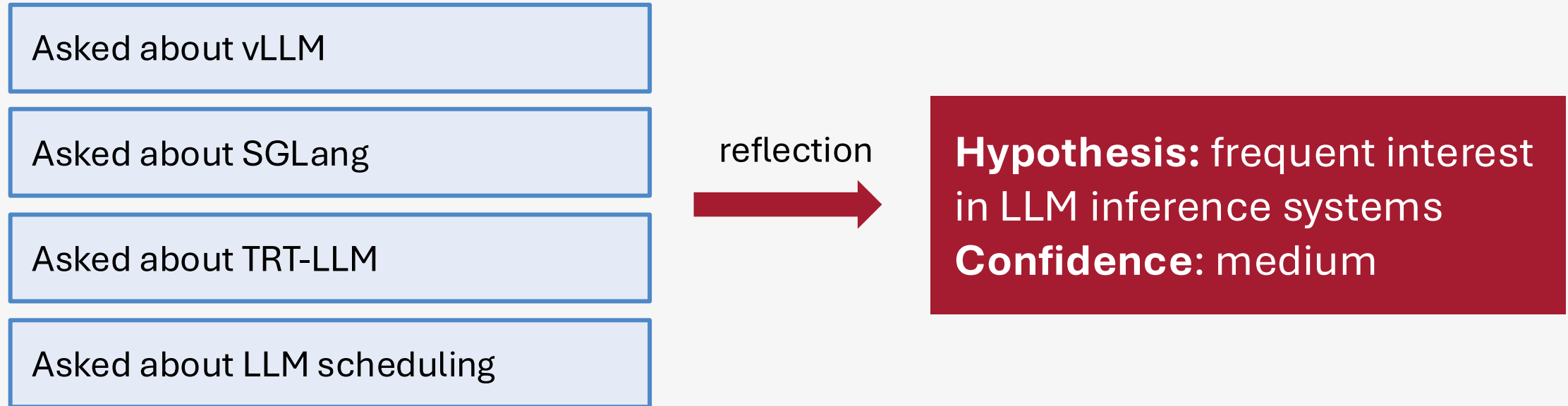
Basic pipeline



Semantic relevance is only one retrieval signal



How to generalize from observations?



When should consolidation happen?

- After enough new evidence, e.g., every N events or context fill
- When facts conflict or repeat
- In a background job when possible

Forgetting is a first-class memory operation

Why forget



- Stale information
- Incorrect or conflicting information
- Privacy and security
- Storage and retrieval cost

Lifecycle controls



- Mark invalid
- Erase permanently
- Expire after a TTL
- Create tiers and evict from active memory



Memory needs valid time and provenance

Valid time: which fact applied when

April 2026

Project serves models with vLLM

superseded



June 2026

Project serves models with SGLang

current

Provenance: where the fact came from

Episode 28

“We moved serving from vLLM to SGLang.”



Project → SERVES_WITH → SGLang



source: Episode 28

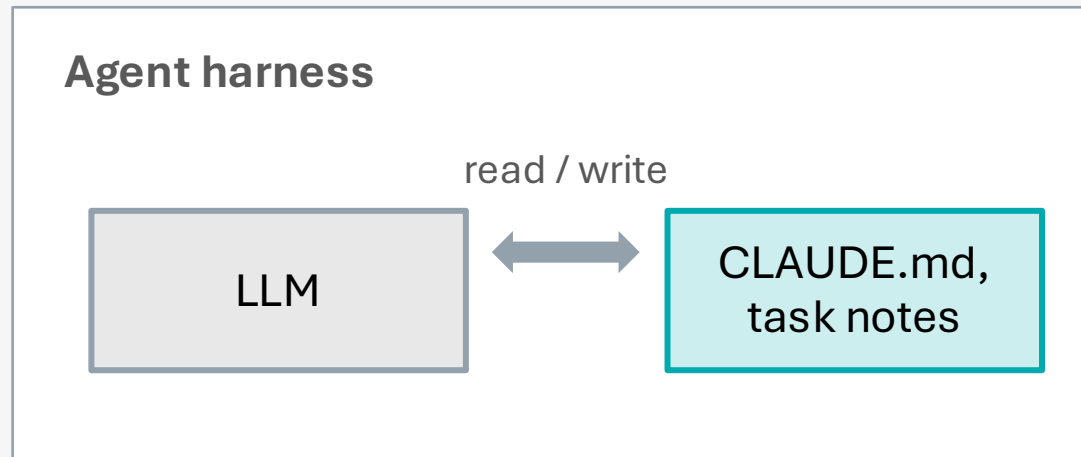
Who owns the memory?

Agent-owned

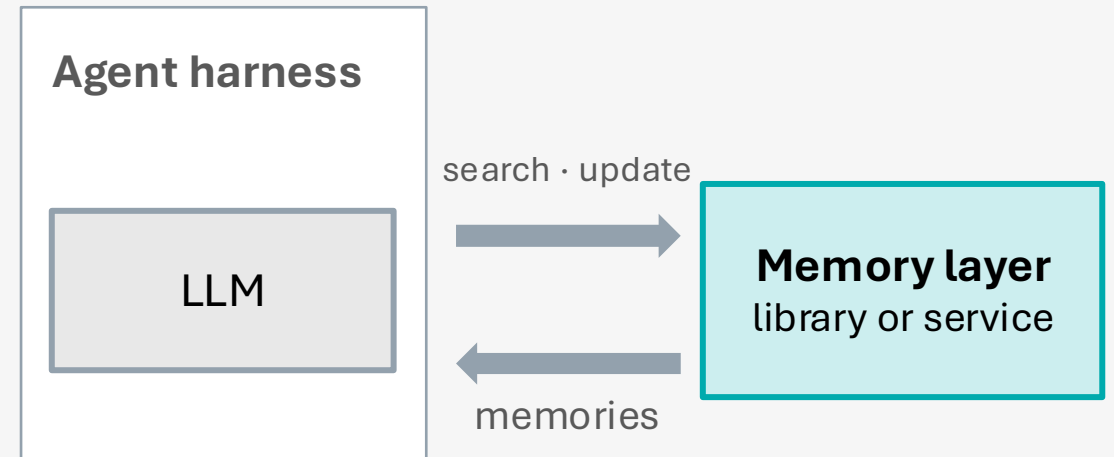
Files in the harness: CLAUDE.md, agent scratchpad, task notes.

Dedicated memory system

A service the agent calls: Mem0, Graphiti, supermemory.



Harness persists memory to disk



Memory is an API, shared across agents and sessions

Dedicate memory systems



Mem0
Fact-centric

Input: **messages**, **scoped** by user, agent, or run.
Retrieval : **vector search** with filters; optional graph memory.
e.g., “Uses SGLang for model serving.”



Graphiti
Temporal knowledge graph

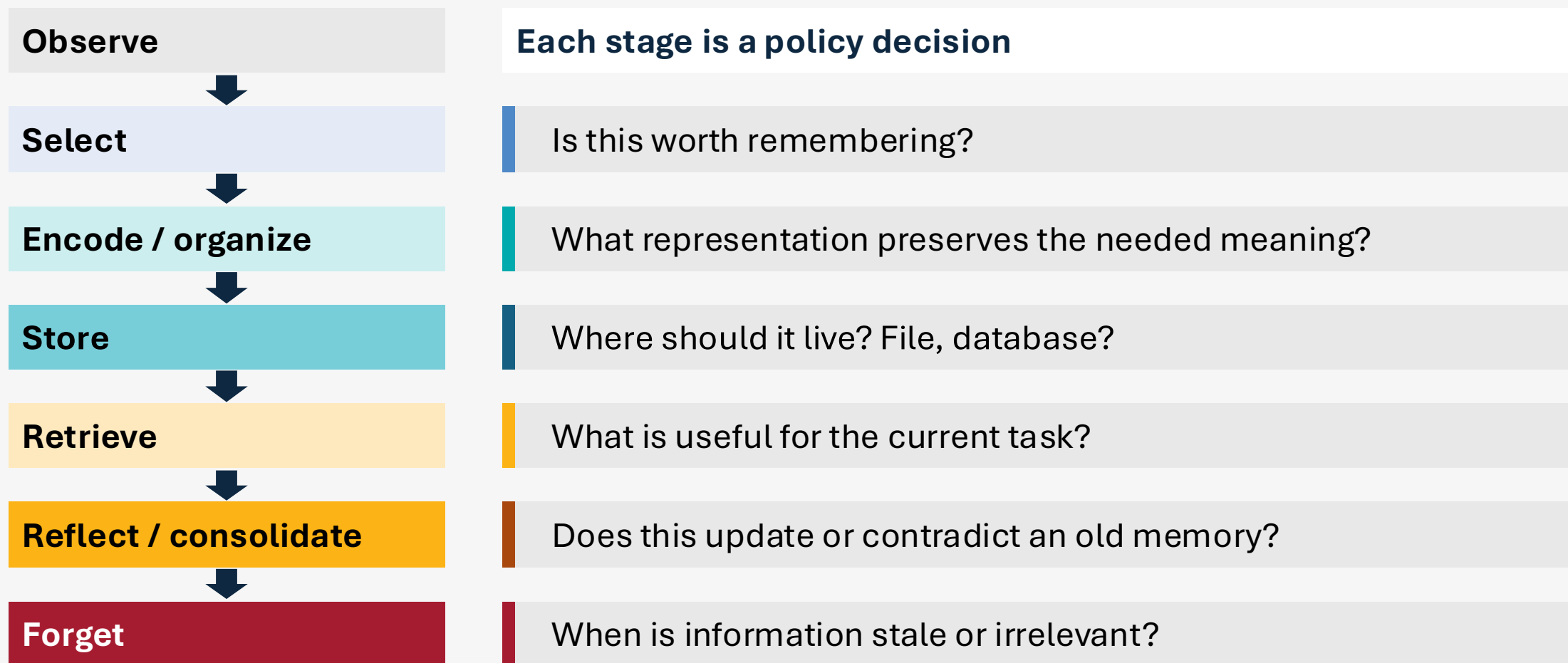
Input: episodes that yield **entities** and **time-aware** facts.
Retrieval: hybrid semantic, keyword and **graph search**.
e.g., uses → SGLang, valid Apr 2026 → present



supermemory
Profile, memory, document

Maintain a **user profile** along with memory and document
Input: conversations linked to source and time.
Retrieval: profile, memory and document search

Agent memory design checklist





Failure modes

Retrieval miss

Relevant memory exists but isn't found.

Context pollution

Too many irrelevant memories enter context.

Memory hallucination

An inference stored as fact: “studying at Harvard” becomes “visiting student at Harvard.”

Staleness

An old fact overrides newer information.

Isolation failure

Memory crosses a user boundary.

Overgeneralization

One event becomes a durable preference.

Persistent prompt injection

Untrusted content returns later as instructions.

Unbounded growth

Size grows with lifetime, so retrieval cost and quality degrade.

How to evaluate agent memory

Retrieval

Precision, recall

Writes and update

Precision, recall, and stale-memory rate

Governance

Deletion compliance and scope isolation

System

Latency, token overhead, and store growth

Agent memory evaluation

Memory component evaluation

LoCoMo

1,540 questions

10 long dialogues

LongMemEval

500 questions

5 memory abilities

BEAM

2,000 questions

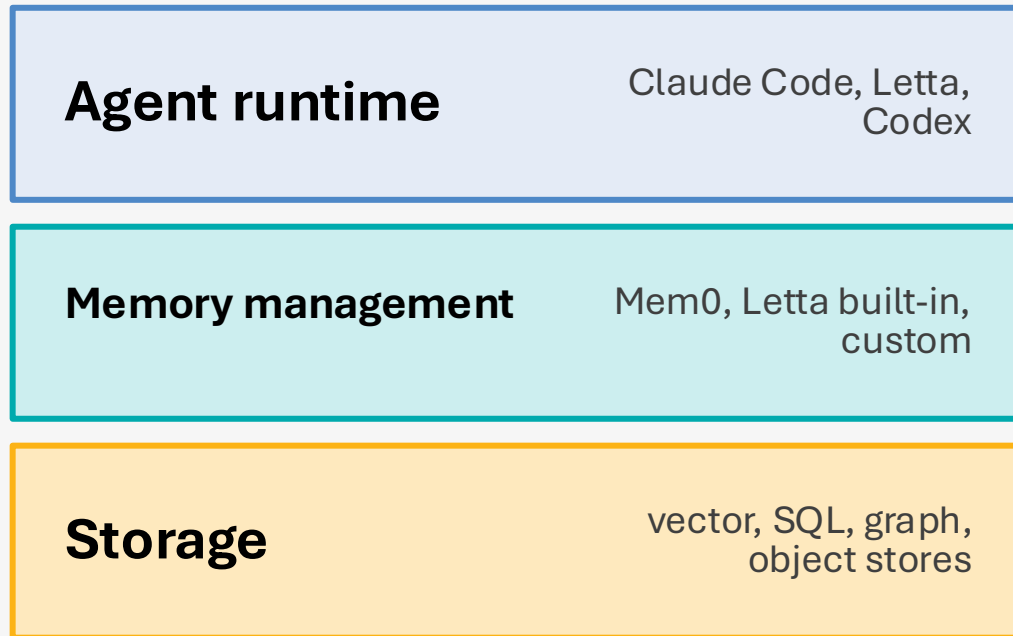
Up to 10M tokens

Mainly test whether an agent can recall and reason over user-assistant histories. They cover **recall, temporal reasoning, updates, preferences and abstention**.

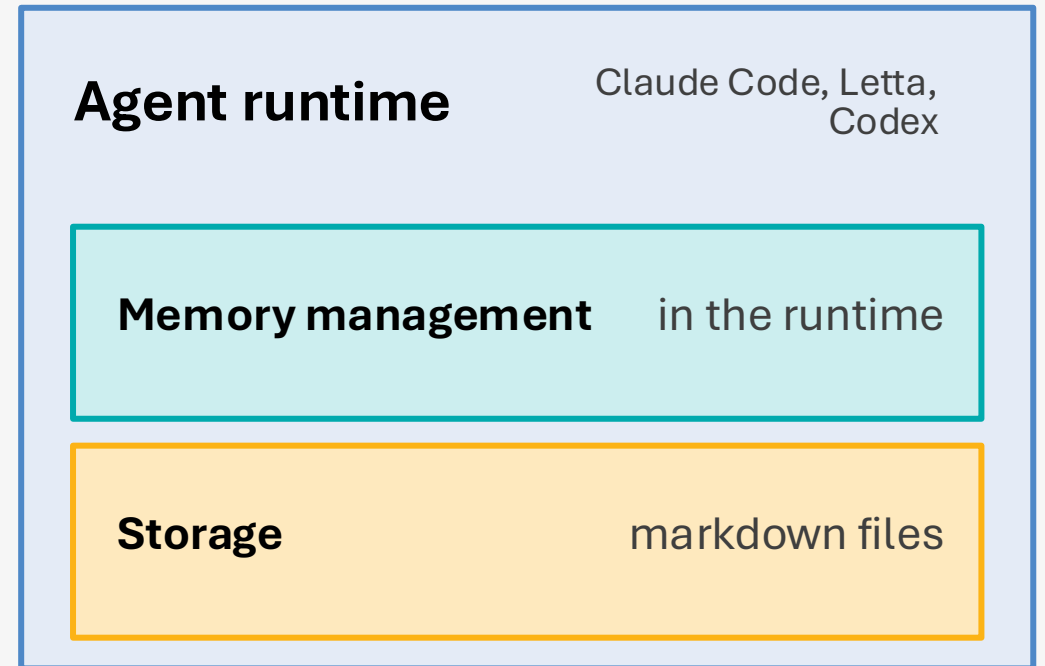
LoCoMo (ACL 2024) · LongMemEval (ICLR 2025) · BEAM (ICLR 2026) · LoCoMo-Refined

The agent memory stack

Separate memory layer



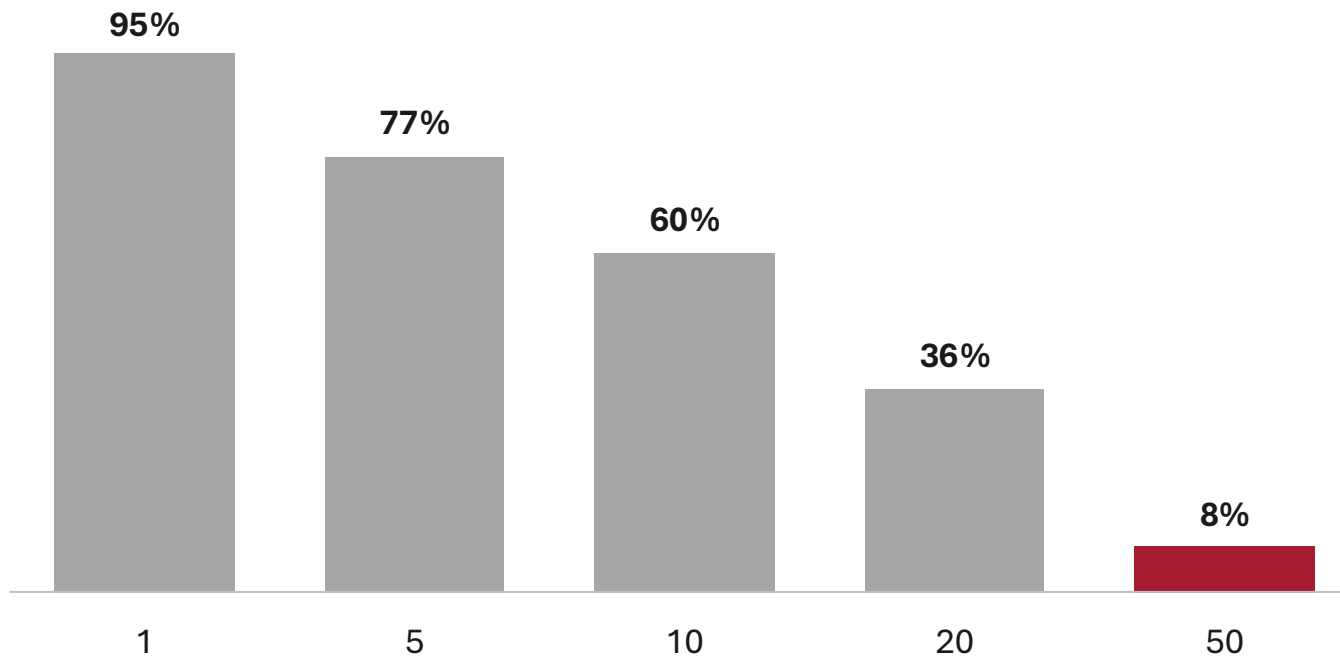
Memory inside the harness



Why is agent reliability a problem?

Chance an agent run is error-free

By number of steps in the trajectory, 95% success per step



Assumes each step succeeds independently with probability 0.95.

Errors compound

1 in 13

50-step runs finish with no errors

- A run succeeds only if **every step** does
- Error propagation makes real runs **worse** than this

Agent reliability

What are the different ways agents fail?

Where does the fault originate?

User / task spec	was the goal unambiguous?	<i>specification failure</i>
Agent harness	context, loop, scheduling	<i>harness failure</i>
LLM	reasoning and planning	<i>model failure</i>
Tool layer	shell, browser, APIs	<i>tool failure</i>
Environment	repo, OS, web, services	<i>environment failure</i>

Task specification

Failure	The spec said	The agent did	The fix
Ambiguous goal	<i>"Fix the login bug"</i>	➔ Fixed the wrong one of three	Name the file, symptom and repro
Proxy objective	<i>"Make the tests pass"</i>	➔ Deleted the failing test	Forbid test edits; review the diff
Unstated constraint	<i>"Speed up search"</i>	➔ Broke the public API	State what must not change
No definition of done	<i>"Clean up the parser"</i>	➔ Never stopped refactoring	Give an exit test and a stop rule
Conflicting goals	<i>"Minimal diff, every edge case"</i>	➔ Silently dropped the edge cases	Rank the goals explicitly

Agent harness

Failure	The harness did	The effect	The fix
Context truncation error	Trimmed the oldest turns	→ Forgot the “keep the API” rule	Pin key rules outside the window
Memory error	Recalled a stale note as fact	→ Edited a file that had moved	Date memories; re-check before use
State corruption error	Kept a half-applied edit	→ Later steps built on broken files	Checkpoint; roll back failed steps
Control loop error	Set no step or cost budget	→ 200 steps, no progress	Cap steps, cost and time
Blocking fan-out	Waited on every sub-agent	→ One slow child stalled the run	Set timeouts; use partial results

Model

Failure	The model reasoned	The agent did	The fix
Hallucination	<i>"requests.fetch() exists"</i>	→ Called a nonexistent function	Check calls against real docs
Instruction violation	<i>"Speed matters more than the API"</i>	→ Changed a signature it was told to keep	Restate constraints; check the diff
Faulty reasoning	<i>"A timeout means the network is down"</i>	→ Added retries to a deadlock	Test the hypothesis before fixing
Reasoning–action mismatch	<i>"I'll only touch parser.py"</i>	→ Rewrote three other files	Compare each action to the plan
Unverified success	<i>"Tests probably pass now"</i>	→ Declared done, ran nothing	Require test output as proof
Long-horizon degradation	<i>"The config still needs fixing"</i>	→ Undid its own fix from step 12	Re-read state; split into short tasks

Tool

Failure	The tool returned	The agent did	The fix
Truncated output	First 100 lines of a log	➔ Missed the error on line 4,000	Return errors first; allow paging
Silent failure	Exit 0, empty result	➔ Assumed there were no matches	Separate "empty" from "failed"
Ambiguous schema	A date with no time zone	➔ Scheduled the job 8 hours off	Use typed, explicit schemas
Flaky call	A timeout on the first try	➔ Declared the service down	Retry with backoff before judging
Misleading error	"Permission denied" on a typo	➔ Retried with sudo	Return precise, actionable errors

Environment

Failure	The environment had	The agent did	The fix
Stale state	Yesterday's cached build	➔ Passed tests on old code	Build clean from a fresh cache
Version mismatch	Python 3.8, not 3.12	➔ Shipped code that crashed	Pin versions in a lockfile
Random failure	Task success depends on environment	➔ Failed bc of leap year	Rerun failures; quarantine flakes
Changed web page layout	A redesigned checkout page	➔ Clicked the wrong button	Re-read the page before acting
Rate limit	HTTP 429 after 50 calls	➔ Stopped mid-migration	Throttle and checkpoint long jobs

Designing for failure

Task specification	Agent harness	Model	Tool	Environment
Write “done” as a test • Acceptance tests up front • List what must not change	Bound every run • Step, cost and time caps • Checkpoint and roll back	Demand evidence • Proof for every claim • Verify • Use stronger models	Fail loudly • Typed schemas • Errors distinct from empty	Pin and isolate • Fresh, pinned sandboxes • Retry flakes before judging

We do not yet understand agents at scale, and there is little formal best practice.

Next class

- Agent evaluation
- Cost-effective agent
- Case studies